

Informe de Compiladores

-

Taller de Diseño de Software 2025

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación

Licenciatura en Ciencias de la Computación

Integrantes del grupo:

- **Jhonatan Calle Galeano**
- **Joaquín Pablo Tissera**

Profesor:

- **Francisco Bavera**

Ayudante de la materia:

- **Agustín Nolasco**

Organización de las tareas

Nuestro primer objetivo para la materia fue seguir las indicaciones dadas en clase, consultando en clase por cómo se debía abordar alguna etapa en particular o despejar dudas de alguna implementación, además de consultar los apuntes dados y ayuda externa por parte de internet.

Al principio los dos integrantes decidimos implementar nuestras propias ideas y unirlos, pero optamos por implementar directamente la primera implementación funcional y seguir trabajando sobre ella.

Con el paso del tiempo, fuimos construyendo las distintas etapas propuestas para el proyecto, de manera tal que Jhonatan se encargaba de plasmar la primera idea (funcional o no) y luego Joaquín revisaba los cambios, aportaba nuevas ideas o corregía detalles de la que ya estaba y refactorizaba el código.

Acordamos utilizar el mismo formateador de C y convenciones similares para mantener legibilidad y uniformidad.

Trabajar de esta manera ayudó a mantener las tareas distribuidas y pudimos sincronizarnos mejor con la materia y otras reponsabilidades.

Primera etapa: Analizador sintáctico y léxico

Decisiones de diseño

Se llevó a cabo la implementación del archivo sintáctico/léxico (**lexicon.l**) y semántico/gramática (**syntax.y**).

Decidimos basarnos en el pre-proyecto para estos dos archivos, pero mejorando en la refactorización del método de entrada (**src/** y **headers/**, **main** y **utils**).

Agregamos los primeros casos de tests y scripts para compilar y ejecutar el compilador.

El hecho de que existan funciones que encapsulan un comportamiento le garantizan extensibilidad al programa, pero sabemos que agrega complejidad de comprensión, es por eso que consideramos necesario agregar comentarios y nombres gnemotécnicos para cada cosa.

Se agregaron flags al momento de ejecutar el compilador obtenido para posteriores etapas.

Asunciones

Asumimos el léxico y la gramática provistas por la documentación del proyecto. En esta etapa no se hicieron cambios respecto a ella, pero sí en etapas posteriores.

Aclaraciones o extensiones

El punto de entrada del compilador viene dada por la función en C llamada **compiler_main**, dentro del archivo **syntax.y**, pasándole como parámetros los argumentos entrantes al momento de llamar a **yyparse()** de bison.

Segunda etapa: Analizador semántico

Decisiones de diseño

Se implementaron las clases **AST** (Abstract Syntax Tree), **Símbolo**, **ScopeNode** y **Scope**, los enumerados **TipoNodo** (nodo de tipo asignación, declaración de variable o método, invocación, suma, resta, identificador, etc), **Tipos** (entero, booleano, void) y **Categoría** (variable o función). Se implementaron funciones de chequeo semántico (errores o warnings de semántica).

Se agregaron las reglas intermedias en el archivo **syntax.y** para ejecutar el código de construcción del árbol y analizador semántico.

Asunciones

Existen variables globales a todo el programa para facilitar su uso conceptual, por ejemplo el scope manejado en el momento y flag-variables usadas al ejecutar el compilador.

Aclaraciones o extensiones

El árbol se construye a medida que se avanza en las reglas dadas por **syntax.y**, es decir, si no existen errores sintácticos (se recorrieron las reglas de la gramática de forma adecuada) el mismo backtracking de las reglas intermedias va a construir el árbol.

Por ende, si la primera regla leída (regla **program** con **R_PROGRAM**) realizó su regla intermedia con éxito, quiere decir que no existen errores sintácticos ni semánticos.

Cambios respecto a etapas anteriores

Algunas reglas ahora son de tipo **AST***, otras de tipo **char***, otras de tipo **int** y otras de tipo enumerado **Tipos** (entero, booleano, void).

Desde el léxico se debe asignar a la variable reservada **yyval** según el número literal o identificador que se lea.

Tercera etapa: Generador de Código Intermedio

Decisiones de diseño

Se implementaron las clases **TAC** (Three Address Code), **TACList** (lista de TACs) y el enumerado **OpCode** (según qué operación se haga; suma, resta, asignación, llamada, return, etc). A partir de esta etapa, para generar código relacionado a una etapa, se debe pedir la etapa corriente o posteriores mediante una flag al momento de ejecutar.

Asunciones

Existe una única TACList y es global a todo el programa.

Aclaraciones o extensiones

La lista de código de tres direcciones se construye luego de construir el árbol sintáctico. Una vez construido el árbol, se pasa la raíz como punto de inicio e insertamos recursivamente a la lista según el tipo de nodo que se esté viendo.

Por ejemplo, si el nodo actual es una declaración de variable, insertamos en la lista de TAC el OpCode de **TAC_ASSIGN** con primer operando la expresión que se va a asignar, segundo operando **NULL** y tercer operando a dónde queremos asignar (resultado).

Se agregó el archivo **assembly_code** para generación de temporales y para la etapa siguiente de generación de código objeto. En la etapa siguiente este archivo define sus cosas propias de assembler y la generación de temporales cambia de lugar.

Cambios respecto a etapas anteriores

Se agregó un método para mostrar el **AST** y más chequeos semánticos.

Se renombró la clase Categoría por MethodCategory.

Se agregó la negación lógica como operador al léxico.

Cuarta etapa: Generador de Código Objeto

Decisiones de diseño

No se añadieron clases nuevas, más sí funciones y conceptos de assembler y código objeto. Si una función tiene más de seis parámetros, usamos seis registros reservados y creamos registros temporales a partir del séptimo.

Asunciones

El código assembler generado fue probado para Linux en distribuciones Ubuntu 22.04 y Mint 22.2.

Aclaraciones o extensiones

El código assembler es generado a partir de la lista de código de tres direcciones. Según la instrucción recibida se genera un cierto código assembler u otro.

Cambios respecto a etapas anteriores

Se agregó la coma y las operaciones menor o igual y mayor o igual como operadores delimitadores al léxico.

Se agregaron chequeos semánticos.

Se agregaron archivos de salida para cada etapa y nuevos scripts para ejecutar el código objeto generado.

Se refactorizaron los métodos del **TAC** para mayor legibilidad y mantenibilidad.

Se pasó la mayor parte del proyecto (nombres de funciones, de variables, comentarios, etc) a inglés para mantener uniformidad en el idioma y mantener una comunicación neutra para otros desarrolladores.

Quinta etapa: Optimizador

Decisiones de diseño

Como pequeña optimización decidimos que el código que aparezca posterior a un return sea avisado (warning), pero tenemos contemplada la posibilidad por ejemplo de lanzar un error al usuario por un error semántico, ya que es código que no va a ejecutarse y puede causar problemas (por ejemplo memory leaks, si se llegara a aceptar que el lenguaje crea y destruye memoria de forma dinámica).

Aclaraciones o extensiones

Esta optimización se añadió junto con la etapa de código objeto.

Discusiones

La discusión de solamente avisar o lanzar un error cuando hay código inalcanzable se da cuando quien escribe el código piensa que tal código inalcanzable en un futuro sí se pueda ejecutar. Por ejemplo:

```
#include <stdio.h>
int f() { return 8; }
void a_function_call() { printf("Called!\n"); }
int driver() {
    switch (f()) {
        case (1): return 1;
        case (2): return 2;
        default: return 3;
    }
    a_function_call();
    return 0;
}
int main() { printf("%d", driver()); }
```

Lanzar un warning en este caso es válido si se conoce que en un futuro la cláusula default se elimine o algún caso no retorne la función.

Este otro ejemplo:

```
#include <stdio.h>
int f() { return 0; }
void a_function_call() { printf("Called!\n"); }
int driver() {
    if (f()) return(1);
    else return(2);
    a_function_call();
    return 0;
}
int main() { printf("%d", driver()); }
```

Similar al caso anterior pero con condicional **if-else** donde si la rama del else no fuese necesaria, el código dejaría de ser inalcanzable.

Ahora bien, para casos donde exista código literalmente luego de un **return**, es código que jamás debería ser alcanzable, ni siquiera en un futuro, porque ya hay un error lógico mucho más grande. Ejemplo:

```
#include <stdio.h>
void a_function_call() { printf("Called!\n"); }
int driver() {
    return 0;
    a_function_call();
}
int main() { printf("%d", driver()); }
```

En estos casos lo ideal sería lanzar un error de código inalcanzable y sería seguro de hacer.

Ya sea por distintos motivos que ese código quedó ahí, realmente no debería estar, por lo que representa el retorno de una función como concepto para la programación.

Problemas conocidos en el proyecto

Recursividad

Si bien no es un problema sino una decisión en el diseño del compilador, sabemos que no soportamos la recursividad. Se intentó conseguirla pero no dió buen resultado.

La idea estaba bien pensada; **insertar el símbolo del nombre de la función luego de sus parámetros y antes que el cuerpo de la función al momento de crear el árbol para que el mismo cuerpo de la función sepa invocarse correctamente a ella misma.**

Sin embargo, por razones de sintaxis/semántica de bison para el manejo de reglas intermedias, adición de complejidad en la solución, tiempo y aval de las especificaciones del proyecto, se decidió no seguir buscando la implementación para este concepto tradicional de la programación como es la recursividad.

Archivos librería / No ejecutables

Este tampoco sería un problema sino otra decisión de diseño del proyecto.

Es posible compilar un archivo con extensión **.ctds** sin una función **main()**, pero no se podrá ejecutar.

Estos archivos los llamaremos librerías, donde van a almacenar funciones que podrán ser llamadas desde otros archivos con un linker apropiado.

Las librerías deben ser compiladas para que puedan ser linkeadas, y el link se hace por medio del código objeto generado.

Reconocimos este problema cuando intentamos ejecutar archivos sin una función main definida, donde el assembler de gcc pedía que existiera un punto de entrada del programa (función reservada main).

Para nuestros casos de prueba de archivos sin función main definida, se les hizo un driver.ctds correspondiente que llamara la función que corresponda de la librería, para que el caso de prueba pueda ser ejecutado.

Conclusión

El desarrollo del compilador presentó un desafío técnico y organizativo que permitió aplicar e integrar varios conocimientos adquiridos durante la carrera.

A lo largo de las distintas etapas logramos construir un sistema coherente, modular y extensible, siguiendo las especificaciones del proyecto.

En cada etapa se tomaron decisiones de diseño importantes, desde la estructura de los árboles de sintaxis abstracta y la tabla de símbolos, hasta la generación de código assembler/objeto funcional. Además, el trabajo en equipo permitió mantener un flujo constante de revisión, corrección y mejora del código, alcanzando una excelente calidad final tanto en legibilidad como en desempeño.

Si bien hubo limitaciones como la no implementación de la recursividad, se tomaron decisiones de diseño justificadas para priorizar estabilidad, claridad y cumplimiento con las especificaciones del proyecto.

En conclusión, el proyecto cumplió con todos los objetivos propuestos, dando a conocer que entendemos el funcionamiento interno de un compilador y fortaleciendo nuestras habilidades en programación, diseño de software y trabajo colaborativo.